DEPARTMENT OF MATHEMATICS & COMPUTER SCIENCE

Faculty of Science and Technology · National University of Lesotho

CS3520

Computer Organisation and Architecture I

Week 7 — The Single-Cycle Processor II: Control, Extension & Its Limits

Mr K. O. Setetemela

Course Coordinator · Department of Mathematics & Computer Science

Academic Year 2026/2027 · Semester A

The Datapath Has No Opinions

Last week: we built a datapath that can carry out four instruction classes — but nothing yet tells it which one it is carrying out. That is the control unit's job.

1	**The Control Problem**	Which switches must be set, and by what
2	**ALU Control**	Two levels of decoding, and why
3	**The Main Control Unit**	Seven signals derived from the opcode
4	**The Control Truth Table**	The whole design on one page
5	**Instruction Walkthroughs**	add, lw and beq through the machine
6	**Extending the Datapath**	Adding a new instruction — what it costs
7	**Why Single-Cycle Fails**	The critical path, and the road to pipelining

THE CONTROL PROBLEM

What Actually Needs Controlling

The datapath is a set of switches. Control decides how each is thrown — and it must decide from the instruction alone.



Multiplexers

Which input wins: ALUSrc, MemtoReg, PCSrc



Memory

Whether to read or write: MemRead, MemWrite



Register File

Whether to commit a result: RegWrite



The ALU

Which operation to perform: ALUOp → ALU control

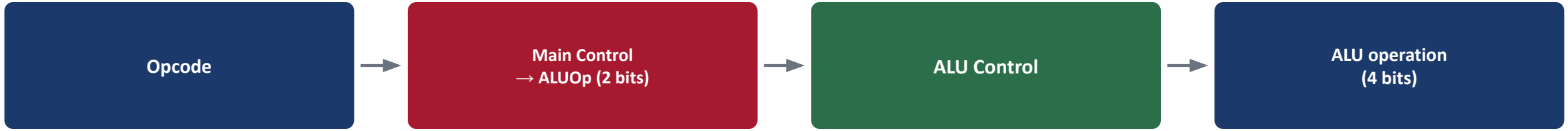
The whole control unit is one combinational function.

Its input is the opcode (plus funct3 and funct7). Its output is every control line, all at once. There is no sequencing, no state, no waiting — which is exactly why one instruction can complete in one cycle.

TWO-LEVEL DECODING

ALU Control

The ALU needs different operations for different classes — but the opcode alone does not say which. Two levels of decoding solve it.



what class of instruction?

a coarse hint

combines ALUOp with funct3 / funct7

the exact operation

Instruction class	ALUOp	Fields consulted	Resulting ALU operation
lw, sw	`00`	don’t care	add — compute the address
beq	`01`	don’t care	subtract — then test Zero
R-type	`10`	funct3 / funct7	add, sub, and, or — as the fields dictate

Why not decode the ALU operation directly from the opcode? Because every R-type instruction shares one opcode. The distinction lives in funct3 and funct7 — so the decision must happen after those fields are available.

Seven Signals

`RegWrite`	Commit a result to the register file
`ALUSrc`	Second ALU operand: register (0) or immediate (1)
`ALUOp`	Coarse ALU class, refined by ALU control
`MemRead`	Read data memory — loads only
`MemWrite`	Write data memory — stores only
`MemtoReg`	Write back the ALU result (0) or the memory value (1)
`Branch`	Combined with ALU Zero to select the next PC

Branch alone does not redirect the PC. PCSrc = Branch AND Zero — the instruction must both be a branch and have its condition satisfied.

THE WHOLE DESIGN ON ONE PAGE

The Control Truth Table

Instruction	RegWrite	ALUSrc	MemRead	MemWrite	MemtoReg	Branch	ALUOp
R-type	1	0	0	0	0	0	10
lw	1	1	1	0	1	0	00
sw	0	1	0	1	X	0	00
beq	0	0	0	0	X	1	01

Read this table as the specification of a logic circuit. X means “don’t care” — the signal has no effect for that instruction, which gives the logic designer freedom to simplify. Notice that sw and beq never write a register, and only lw reads memory. Every row is a design decision you could defend.

PUTTING IT TOGETHER

Three Instructions, One Machine

`add x1, x2, x3`

1. Fetch instruction; PC + 4 computed in parallel
2. Read x2 and x3 from the register file
3. ALU adds them (ALUOp = 10, ALUSrc = 0)
4. Memory stage idle — but still traversed
5. Result written to x1 (RegWrite = 1, MemtoReg = 0)

`lw x1, 8(x2)`

1. Fetch instruction; PC + 4 computed in parallel
2. Read x2; sign-extend the 12-bit offset
3. ALU adds base + offset (ALUSrc = 1)
4. Data memory read at that address (MemRead = 1)
5. Memory value written to x1 (MemtoReg = 1)

`beq x1, x2, L`

1. Fetch instruction; PC + 4 computed in parallel
2. Read x1 and x2; sign-extend and shift the offset
3. ALU subtracts (ALUOp = 01); Zero is tested
4. Branch target computed by the second adder
5. PCSrc = Branch AND Zero selects the next PC

DESIGN EXERCISE

Extending the Datapath

Suppose we must add addi to the machine. What does it cost? Work through the question in this order — it is the order a real design review follows.

1	**What does the instruction need?**	One register, one sign-extended immediate, an ALU add, a register write.
2	**Does the datapath already provide it?**	Yes — lw already routes an immediate to the ALU through ALUSrc, and R-type already writes back an ALU result.
3	**What new hardware is required?**	None. This is the pay-off of a regular instruction set.
4	**What new control is required?**	One new row in the truth table: RegWrite = 1, ALUSrc = 1, MemtoReg = 0, MemRead = MemWrite = Branch = 0, ALUOp = 00.

Now ask the harder version: what would it cost to add a multiply instruction, or an instruction that reads two memory locations? Not every extension is free.

THE RECKONING

The Critical Path

The clock must be slow enough for the slowest instruction. For this machine, that is lw — it is the only one that uses every stage.




```
Tc = t(PC) + t(Imem) + t(RFread)
     + t(ALU) + t(Dmem) + t(RFsetup)
```

CPI = 1, which sounds ideal. But the clock period is dreadful — and CPU time is the product of both.

An add finishes its real work long before the clock edge arrives — and then waits. That wasted time, multiplied across every instruction in every program, is what the next design must recover.

WORKED EXERCISE

Costing a Real Design

Bohlale is designing a 32-bit single-cycle RISC-V processor. Her library gives these delays. How long will a 10-million-instruction program take?

PC (clock-to-Q)	**30 ps**
Memory access	**250 ps**
Register file read	**150 ps**
ALU	**200 ps**
Multiplexer	**25 ps**
Register file setup	**20 ps**

```
Tc = 30 + 2(250) + 150 + 200 + 25 + 20
   = 925 ps

CPI = 1            (single-cycle)
T   = IC x CPI x Tc
    = 10,000,000 x 1 x 925 ps
    = 9.25 ms
```

Now interrogate the result. Memory is accessed twice on the critical path and accounts for 500 of those 925 picoseconds — more than half. Which single component would you improve first, and what would that do to the other instructions that never touch memory at all?

THINK ABOUT IT

CPI = 1 is the best possible figure — and this machine achieves it. Yet no processor has been built this way in decades.

If a design hits the ideal on one metric and is still abandoned, what does that tell you about optimising a single number? Which term of CPU Time = IC × CPI × Tc did we sacrifice, and would you accept CPI > 1 to get it back?

Discuss with a neighbour for two minutes before we continue.

THIS WEEK

Lab 6 & Tutorial Set 3

Build the control unit

Implement the truth table as combinational logic in Logisim-evolution.

Wire it to your Week 6 datapath

Drive ALUSrc, MemtoReg, RegWrite and the rest from the opcode.

Extend it

Add addi to your machine — one new truth-table row, no new hardware.

Verify

Run a short instruction sequence and confirm each control line behaves as the table predicts.

TUTORIAL SET 3

- Complete the control truth table for jal
- Compute Tc for a modified datapath
- Identify the critical path when memory is made twice as fast

---

Mid-semester feedback poll opens this week on the course LMS. It is anonymous, it takes two minutes, and the results — with my response — come back to you in class.

CHECKPOINT · CLO3

Key Takeaways

1. Derive every control signal for a given instruction, and justify each from what the datapath must do.
2. Explain why ALU control is decoded in two levels rather than one.
3. Extend the datapath and control to a new instruction, and state honestly what hardware it costs.
4. Compute the clock period from component delays, and identify the critical path.

COMING UP — WEEK 8

Pipelining I — The Pipelined Datapath

Recovering the wasted time: five stages, structural and data hazards, and forwarding



Reading: Patterson & Hennessy, RISC-V Edition, §4.4